

Capacity Estimation, Clustering and Load Balancing

A complete study guide: concepts, business examples,
technical Q&A and scenario-based questions

Based on lecture slides 56-57

Course: Scalable and Distributed Systems | Dept. of CSE, University of Global Village

Prepared for: Emon Hossain

00

Overview and Topic Map

What the four slides cover and how the ideas connect

The slides come from one lecture on **designing systems that stay fast and available as demand grows**. They cover four connected ideas: what to measure (key metrics), how to plan (the capacity estimation process), and two techniques to deliver the planned capacity (clustering and load balancing).

Slide	Topic	Core question it answers
56	Understanding Capacity Estimation	What do we measure, and why does getting the number wrong cost money?
57	Steps in Capacity Estimation	How do we go from a business forecast to an actual hardware/cloud plan?
57	Clustering and Load Balancing	Why are these two techniques essential for scalable, available apps?
58	Load Balancing (vs Clustering)	What does a load balancer give us that a cluster alone does not?

The big picture in one sentence

Capacity estimation tells you *how much* you need; **clustering** gives you *many machines working as one system*; **load balancing** decides *which machine gets each request*. Together they let a business serve more users, survive failures and avoid wasting money.

Note on the slide numbers: the page footer on the third and fourth screenshots appears cropped, so slide numbers there are inferred from the sequence; the content itself is unaffected.

01 Understanding Capacity Estimation

Slide 56 - key metrics and the five-phase lifecycle

Capacity estimation is the discipline of predicting how much computing resource (CPU, memory, disk, network, number of servers) a system needs to meet its performance and availability goals, now and in the future. It sits between two failure modes: buying too much (waste) and buying too little (outages).

Key metrics

Metric	Meaning (from the slide)	Typical unit	Practical note
Throughput	Transactions or requests handled per second	TPS / RPS	The headline capacity number. Measure at peak, not average.
Latency	Time to complete one transaction or request	ms	Often reported as p50, p95, p99. Tail latency (p99) hurts users most.
Response time	Total time a user waits for a response	ms / s	Latency plus queueing, network and rendering. This is what the user feels.
Load and concurrency	Number of concurrent users or operations	users / connections	Drives memory, thread pools and connection limits.
Utilization	Percentage of capacity currently used	%	Healthy targets are usually 60-75% at peak, leaving headroom.
Business impact	Cost of over-provisioning vs risk of under-provisioning	currency / lost revenue	Turns engineering numbers into a budget decision.

How the metrics relate to each other

Latency vs response time. Latency is how long the work itself takes; response time is what the user experiences, including waiting in queues. When a server is lightly loaded the two are almost equal. When it is heavily loaded, queueing dominates and response time explodes even though the work per request is unchanged.

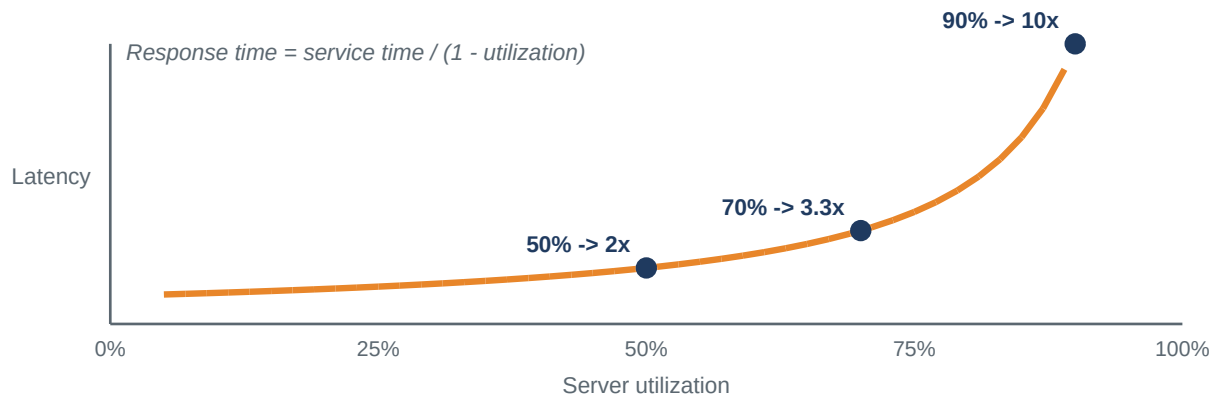
Little's Law links concurrency, throughput and response time. It is one of the most useful capacity formulas:

$L = \lambda \times W$
 L = average number of requests in the system (concurrency)
 λ = arrival rate (throughput, requests per second)
 W = average time each request spends in the system (response time)

Example: a service receiving 500 requests/second with a 200 ms average response time holds $500 \times 0.2 = 100$ **concurrent requests** at any moment. Your thread pool, DB connections and memory must be sized for at least that, plus headroom.

Why high utilization is dangerous

Utilization and latency are not linear. For a simple single-queue model, response time is roughly **service time / (1 - utilization)**. A request that takes 100 ms to process takes about 200 ms at 50% utilization, about 333 ms at 70%, and about 1,000 ms at 90%. This is why engineers keep a safety margin instead of running machines near 100%.

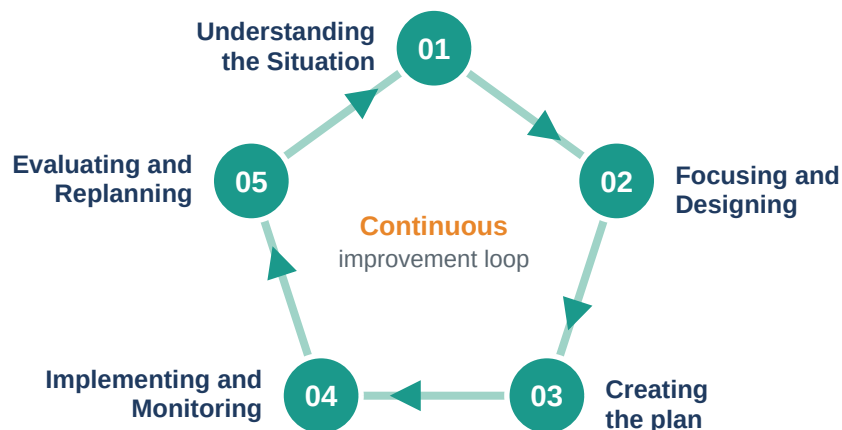


Business impact: the two risks

Over-provisioning: idle servers still cost money (cloud bills, power, licences). **Under-provisioning:** slow pages and outages cost sales, reputation and sometimes regulatory penalties. Capacity estimation exists to find the cheapest point that is still safe.

The five-phase capacity estimation lifecycle (diagram on slide 56)

The ring diagram on the slide shows capacity estimation as a **cycle**, not a one-off task.



Phase	What happens	Example output
01 Understanding the situation	Learn the business goals, current system, expected users and events	'Expect 3x traffic during the Eid sale'
02 Focusing and designing	Choose what to measure and design the estimation approach	Pick RPS and p95 latency as the targets
03 Creating	Build the capacity model or plan using data and assumptions	Spreadsheet or model: 10 servers needed at peak
04 Implementing and monitoring	Provision resources, deploy dashboards and alerts	Auto-scaling rules, Grafana dashboards
05 Evaluating and replanning	Compare predictions with reality, then start the cycle again	'Actual peak was 20% higher; adjust the model'

02 Steps in Capacity Estimation

Slide 57 - from requirements to growth forecast

The slide lists six steps. Below, each one is explained with what to do and what can go wrong.

Step	What to do	Common mistake
1. Define requirements	Identify the expected workload, peak traffic and availability needs (for example 99.9% uptime, p95 under 300 ms).	Planning for average traffic instead of peak.
2. Analyze historical data	Mine logs and monitoring for daily/weekly/seasonal patterns and growth trends.	Ignoring seasonality such as festivals, salary days and promotions.
3. Model the system	Build the model in three parts (see below).	Treating all requests as equal cost.
4. Conduct load testing	Run load and stress tests to validate the model and find bottlenecks.	Testing with unrealistic data or without the database under load.
5. Provision resources	Calculate resources for the projected capacity with a margin for peak usage .	Zero headroom, so one failure causes an outage.
6. Estimate growth	Forecast workload growth from business expectations (marketing plans, new markets).	Forecasting once and never revisiting.

Step 3 in detail: modeling the system

Sub-step	Description	Example
Workload modelling	Characterize the types and intensity of workloads, e.g. read-heavy vs write-heavy.	A news site is 98% reads; a payment app has a large share of writes.
Resource consumption modelling	Quantify CPU, memory and disk I/O used by each workload.	Checkout request: 40 ms CPU, 15 MB RAM, 3 DB writes.
Concurrency and scaling factors	Include concurrency and examine how each resource is affected as load grows.	Doubling users may triple DB locks even if CPU only doubles.

Worked example: sizing a web application

A startup expects **2 million daily active users**, each making about **20 requests per day**. Peak traffic is about **5 times** the daily average. One application server can handle **400 requests/second**, but the team wants to run at no more than **70% utilization**. They also want one spare server (N+1) and expect **50% growth** within a year.

Step	Calculation	Result
Requests per day	$2,000,000 \times 20$	40,000,000
Average RPS	$40,000,000 / 86,400$	about 463 RPS
Peak RPS	463×5	about 2,315 RPS

Step	Calculation	Result
Usable capacity per server	400×0.70	280 RPS
Servers for today's peak	$2,315 / 280 = 8.27$, round up	9 servers
Add N+1 redundancy	$9 + 1$	10 servers
Servers after 50% growth	$(2,315 \times 1.5) / 280 = 12.4$, round up, then +1	14 servers

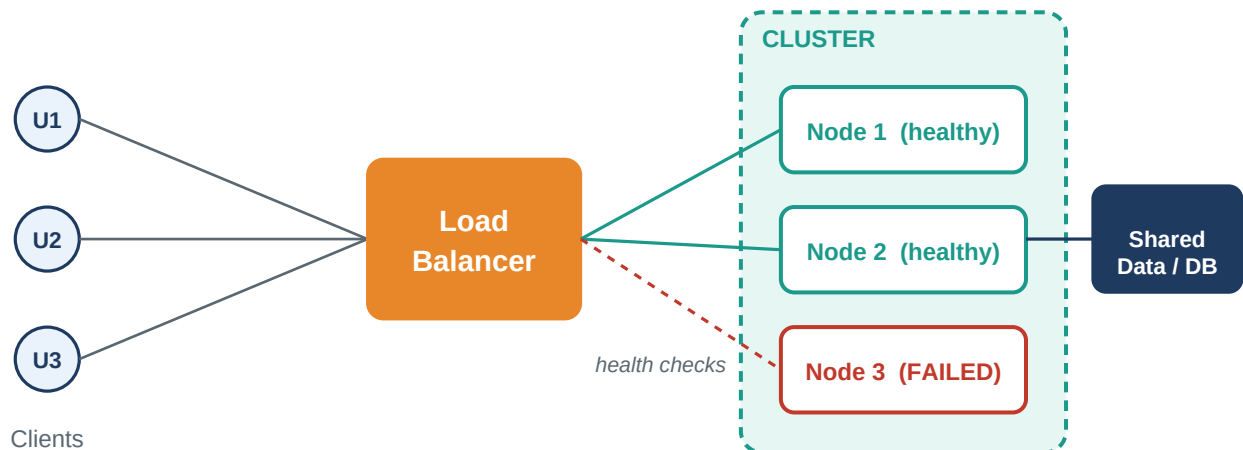
Takeaway

Provision **10 servers now** and have a plan (or auto-scaling) to reach **14**. Re-run the numbers after load testing, because the 400 RPS per server figure is an assumption that must be measured.

03 Clustering and Load Balancing

Slide 57 (second part) - why modern applications depend on both

A **cluster** is a group of servers (nodes) that cooperate so that clients see them as one logical system. A **load balancer** sits in front and distributes requests among the nodes. The slide's diagram shows users sending requests to a load balancer, which forwards each one to Node 1 or Node 2 inside the cluster.



The four benefits listed on the slide

Benefit	Meaning	How it works in practice
High availability	If one server goes down, others take over, minimizing downtime.	Health checks detect the failed node and remove it from rotation; traffic flows to survivors.
Scalability	Adding nodes lets the app handle more users and data without degrading.	Horizontal scaling: add more identical nodes behind the balancer.
Fault tolerance	The cluster keeps operating even when individual nodes fail.	Replicated data, redundant components and no single point of failure.
Resource management	Workloads are spread over nodes so no single node becomes a bottleneck.	Scheduler or balancer routes by load, connections or capacity.

Cluster types

Type	Idea	Typical use
Active-active	All nodes serve traffic at once	Web/API tiers, stateless services
Active-passive (failover)	Standby node takes over only when the primary fails	Databases, legacy apps, licensed software
Shared-nothing	Each node owns its data partition (sharding)	Large databases such as Cassandra or sharded MongoDB
Compute cluster	Many nodes cooperate on one big job	Kubernetes, Hadoop and Spark, HPC

Important design concerns

- **Stateless services:** keep session data in Redis or a database instead of node memory so any node can serve any request.
- **Split-brain:** if nodes lose contact with each other, both may believe they are the leader. Quorum (majority voting) prevents this, which is why clusters often use an odd number of nodes (3, 5).
- **Data consistency:** replicating data across nodes forces trade-offs between consistency, availability and latency (the CAP theorem).
- **The balancer itself:** a single load balancer becomes a single point of failure, so production setups run it in pairs or use a managed service.

04 Load Balancing

Slide 58 - benefits, algorithms and comparison with clustering

Load balancing distributes incoming traffic across multiple servers so that none is overwhelmed. The slide's image contrasts **load balancing** (one entry point fanning out to several targets) with **clustering** (several servers grouped as one system).

Benefit (slide)	Explanation	Business meaning
Efficient resource utilization	Traffic is spread so no single server is overwhelmed.	Pay for servers that are actually used.
Improved performance	Requests are answered faster, improving user experience.	Lower bounce rate, higher conversion.
Redundancy	If a server fails, traffic is redirected to operational servers.	Fewer incidents and less lost revenue.
Scalability	Add servers to the pool to handle growing traffic seamlessly.	Handle campaigns and viral spikes.

Load balancing vs clustering

Aspect	Load balancing	Clustering
Main purpose	Distribute incoming requests	Make several nodes work together as one system
Focus	Traffic routing	Coordination, shared state, failover
Nodes know each other?	Not necessarily; the balancer tracks them	Yes; nodes communicate (heartbeat, replication)
Failure handling	Stops sending traffic to failed targets	Promotes a new leader, restarts or reschedules work
Example tools	NGINX, HAProxy, AWS ALB/ELB, Cloudflare	Kubernetes, Redis Cluster, Galera, Pacemaker
Relationship	Usually used together : the balancer fronts the cluster	Cluster provides the pool the balancer sends to

Common load balancing algorithms

Algorithm	How it picks a server	Best for
Round robin	Takes servers in turn	Equal servers, similar request cost
Weighted round robin	Stronger servers get more turns	Mixed hardware
Least connections	Sends to the server with fewest active connections	Long-lived or uneven requests
Least response time	Chooses the fastest-responding server	Latency-sensitive apps
IP hash / consistent hash	Same client or key always maps to the same server	Sticky sessions, caches
Random (power of two choices)	Pick two random servers, use the less loaded	Very large fleets

Layer 4 vs Layer 7

Layer 4 (transport) balancers route on IP and port; they are very fast and protocol-agnostic. **Layer 7 (application)** balancers understand HTTP, so they can route by URL path (/api to the API pool, /img to the media pool), terminate TLS, inspect headers and cookies, and apply rate limits, at the cost of more CPU.

A minimal NGINX example (least-connections load balancing):

```
upstream app_pool {
    least_conn;
    server 10.0.0.11:3000 max_fails=3 fail_timeout=10s;
    server 10.0.0.12:3000 max_fails=3 fail_timeout=10s;
    server 10.0.0.13:3000 backup;
}
server {
    listen 80;
    location / { proxy_pass http://app_pool; }
}
```

05 Real-Life Business Examples

How companies apply these ideas (illustrative scenarios)

The scenarios below are simplified teaching examples. Real companies use more complex designs, but the principles are the same.

E-commerce flash sale (Daraz-style 11.11 or Eid campaign)

Situation: A marketplace sees normal traffic of 3,000 requests/second, but sales launch drives it to 25,000+ in minutes.

How the concepts apply: **Capacity estimation:** analyze last year's campaign logs, model checkout as the costliest workload, load test at 1.5x the forecast. **Clustering:** separate clusters for catalog (read-heavy), cart/checkout (write-heavy) and search. **Load balancing:** layer 7 balancer with path-based routing, auto-scaling groups pre-warmed an hour before the sale.

Business impact: Under-provisioning means a crashed checkout during the most valuable hour of the year. Over-provisioning all year wastes money, so they scale up only for the campaign window.

Mobile financial services (bKash/Nagad-style)

Situation: Payments must work 24/7. Salary day and festival cash-out cause sharp peaks.

How the concepts apply: **Clustering:** active-active application nodes across data centers; database in a primary-replica or quorum cluster. **Load balancing:** redundant balancers with health checks; transaction retries are idempotent. **Capacity:** peak TPS is the key metric, with large headroom because failures damage trust.

Business impact: Every minute of downtime means failed payments, support calls and potential regulatory attention. High availability is a business requirement, not a luxury.

Video streaming (Netflix-style)

Situation: Millions of viewers stream simultaneously, with huge evening peaks.

How the concepts apply: **Load balancing** at several levels: DNS/geographic routing to the nearest region, then balancers in front of service clusters. **Clustering:** hundreds of microservices each running as a cluster. **Capacity:** forecasts based on viewing history and new-release schedules; extra capacity for big premieres.

Business impact: Buffering drives customers away. Regional failover keeps the service up even if an entire data center fails.

Ride-hailing (Pathao/Uber-style)

Situation: Demand spikes in rain, rush hour and events; location updates arrive constantly.

How the concepts apply: **Workload modelling:** location pings are write-heavy and high-volume, ride search is read-heavy. **Clustering:** sharded location store partitioned by city. **Load balancing:** consistent hashing to keep a city's data on predictable nodes.

Business impact: Slow matching means drivers and riders cancel. Capacity per city is forecast separately because load differs.

Online ticketing and exam result portals

Situation: A bus/train/movie ticket sale or a national exam result opens at a fixed time and everyone arrives at once.

How the concepts apply: **Capacity estimation:** peak concurrency is estimated from the number of eligible users.

Mitigation: a waiting-room queue, caching of static pages at CDN, read replicas for result lookups, rate limiting at the balancer.

Business impact: Without planning the site collapses at the exact moment of highest demand, damaging public trust.

SaaS startup growing from 1,000 to 100,000 users

Situation: A small team begins with one server and a database.

How the concepts apply: **Stage 1:** single server. **Stage 2:** add a load balancer and two app nodes. **Stage 3:** database read replicas and cache. **Stage 4:** clusters per service with auto-scaling. Capacity estimation is repeated at each growth stage (the replanning loop on slide 56).

Business impact: Scaling only when needed keeps burn rate low, while load tests prevent growth from turning into outages.

06 Technical Questions and Answers

Concept and calculation questions for exams and interviews

Q1. What is capacity estimation and why is it needed?

Capacity estimation predicts the resources a system needs to meet its performance and availability goals at expected load. It prevents under-provisioning (slow or failing service) and over-provisioning (wasted money), and gives a defensible basis for budgets and scaling plans.

Q2. Differentiate between latency and response time.

Latency is the time to complete a transaction or request. Response time is the total time the user waits, including queueing delays, network transfer and client rendering. Under heavy load response time grows much faster than service time because of queueing.

Q3. Differentiate between throughput and concurrency. How are they related?

Throughput is completed requests per unit time (RPS/TPS). Concurrency is the number of requests in progress at the same moment. Little's Law connects them: $\text{concurrency} = \text{throughput} \times \text{response time}$.

Q4. A service handles 800 requests/second with 250 ms average response time. How many concurrent requests exist?

By Little's Law, $L = 800 \times 0.25 = \mathbf{200 \text{ concurrent requests}}$. Connection pools and thread pools should be sized above this with headroom.

Q5. Why should you not run servers at 95% utilization?

Queueing delay rises sharply near saturation (response time is roughly service time / (1 - utilization)). At 95% the delay is about 20 times the service time. There is also no room for traffic bursts, failover or garbage-collection pauses. Typical peak targets are 60-75%.

Q6. Explain workload modelling and resource consumption modelling.

Workload modelling characterizes the mix and intensity of requests (for example 90% reads, 10% writes; peak at 8 pm). Resource consumption modelling measures how much CPU, memory, disk I/O and network each request type uses. Together they let you compute total resource demand for any traffic level.

Q7. Why is load testing a required step?

Models rest on assumptions. Load testing validates them with real measurements, exposes bottlenecks (database locks, connection limits, slow queries) and finds the breaking point. Stress tests go beyond expected load to see how the system fails and recovers.

Q8. What is the difference between horizontal and vertical scaling?

Vertical scaling (scale up) adds CPU/RAM to one machine; it is simple but has a hardware ceiling and remains a single point of failure. Horizontal scaling (scale out) adds more machines; it needs clustering and load balancing but gives near-linear growth and fault tolerance.

Q9. What is the difference between clustering and load balancing?

Clustering groups servers so they work together as one system with shared coordination and failover. Load balancing distributes incoming requests across servers. A balancer routes traffic; a cluster provides the cooperating pool. They are usually combined.

Q10. Compare round robin and least connections.

Round robin gives servers equal turns, which works well when requests are similar in cost and servers are equal. Least connections sends new requests to the server with the fewest active connections, which handles uneven or long-lived requests (uploads, WebSockets) better.

Q11. What are sticky sessions and what is the better alternative?

Sticky sessions pin a user to one server (via cookie or IP hash) because session data lives in that server's memory. The downside is uneven load and lost sessions on failure. The better design is stateless servers with sessions in a shared store such as Redis or in signed tokens (JWT).

Q12. What is a health check and why is it essential?

A health check is a periodic probe (HTTP endpoint, TCP connect) the balancer sends to each server. Unhealthy servers are removed from rotation automatically and re-added when they recover. Without it, users would keep being routed to dead servers.

Q13. What is split-brain and how is it prevented?

Split-brain happens when a network partition makes two parts of a cluster each think they are the active one, risking conflicting writes. It is prevented with quorum (a majority must agree), fencing, and an odd number of voting nodes.

Q14. Explain availability math: how does redundancy improve uptime?

If one server is 99% available and servers fail independently, two in parallel are available unless both fail: $1 - (0.01 \times 0.01) = 99.99\%$. In practice shared dependencies (the load balancer, the database, the network) reduce this, so true availability is limited by the weakest shared component.

Q15. What is the difference between Layer 4 and Layer 7 load balancing?

Layer 4 routes on IP address and port, which is fast and works for any TCP/UDP service. Layer 7 inspects HTTP data (path, headers, cookies), enabling path-based routing, TLS termination, caching and rate limiting, but uses more CPU.

Q16. Why is the load balancer itself a risk and how do you mitigate it?

It can be a single point of failure and a bottleneck. Mitigate with an active-passive or active-active pair (virtual IP / VRRP), DNS-level distribution, or a managed cloud balancer that is itself redundant across zones.

07 Scenario-Based Questions and Answers

Apply the concepts to realistic problems

Q17. Your e-commerce site crashes every time a sale starts. CPU on the app servers is at 100% while the database is at 30%. What do you do?

Diagnosis: the application tier is the bottleneck, not the database.

Immediate fix: add app nodes behind the load balancer (scale out) and enable auto-scaling on CPU or request rate. Pre-warm instances before the sale.

Underlying work: profile the code (hot endpoints, inefficient serialization), add caching for product pages, and move static assets to a CDN.

Prevent recurrence: run a load test at 1.5x expected peak before every campaign and add alerts at 70% utilization.

Q18. A server in a three-node cluster crashes at 2 a.m. What should happen automatically, and what should the team verify next morning?

Automatically: the health check fails, the balancer stops routing to the node, the remaining two nodes absorb traffic, and an alert is raised. An orchestrator may restart or replace the node.

Check the margin: with 2 of 3 nodes the survivors carry 50% more load each. If they were above about 66% utilization before, they are now overloaded. This is why N+1 capacity matters.

Next morning: review logs for the crash cause, confirm data replication caught up, and confirm the replacement node rejoined correctly.

Q19. Management wants to cut the cloud bill. Traffic peaks 8 pm to 11 pm, and servers sit at 15% utilization for the rest of the day. What do you propose?

Use **auto-scaling** with scheduled and metric-based policies so the fleet shrinks off-peak and grows before the evening peak.

Buy reserved or committed capacity only for the always-needed baseline; use on-demand or spot instances for peaks.

Risk control: keep a minimum of 2 nodes for availability, set a scale-out threshold around 60-65%, and test that scale-out is fast enough (warm-up time) so users do not feel it.

Business framing: present the savings alongside the risk of under-provisioning (the slide's business impact point).

Q20. Users complain they are randomly logged out. You recently added a second app server behind a round-robin balancer.

Cause: sessions are stored in each server's memory. A user logged in on server A is sent to server B on the next request, which has no session.

Quick fix: enable sticky sessions (cookie-based affinity).

Proper fix: externalize sessions to Redis or use signed JWT tokens so servers are stateless. This also makes failover and autoscaling safe.

Q21. Estimate the number of servers needed for a school-admission portal expecting 300,000 applicants over a 3-day window, with a daily peak hour carrying 30% of that day's traffic. Each applicant makes about 40 requests. One server handles 150 RPS at the 70% target.

Assume traffic is spread evenly over 3 days: 100,000 applicants per day.

Requests per day: $100,000 \times 40 = 4,000,000$. Peak hour carries 30%: 1,200,000 requests in 3,600 s = about **333 RPS**.

Usable capacity per server: $150 \times 0.70 = 105$ RPS. Servers: $333 / 105 = 3.2$, round up to **4**. Add N+1: **5 servers**.

Sanity check: deadlines usually cause a rush on the last day, so a real plan should test a 2-3x higher peak and use auto-scaling.

Q22. During a rolling deployment, the new version has a bug and returns HTTP 500. How can load balancing limit the damage?

Deploy to one node first (**canary**) and send only a small percentage of traffic to it (weighted routing).

Health checks and error-rate monitoring detect the problem; the balancer removes the canary or the pipeline rolls back automatically.

Alternatively use **blue-green deployment**: switch the balancer between two full environments and switch back instantly on failure.

Q23. Your database cluster is healthy, but the application is slow because a single heavy query blocks the primary node. Would adding more app servers help?

No. The bottleneck is the database, so more app servers would send even more load to the same primary.

Optimize or index the query, add a cache, and move read traffic to **read replicas** (load balancing reads across replicas).

If writes are the issue, consider sharding or queueing writes. This shows why capacity must be estimated per tier, not just for web servers.

Q24. A national exam result site is expected to receive 2 million visits within the first 10 minutes. How would you design for it?

Cache the results lookup page shell on a **CDN**; make the lookup API read-only and backed by a read-optimized store with replicas.

Put a layer 7 balancer with **rate limiting** and a waiting-room page that admits users at a rate the backend can handle.

Estimate: $2,000,000 / 600$ s is about 3,333 RPS average, so plan for roughly 2-3x that at the instant of release. Load test at that level.

Prepare a static fallback page and an incident runbook, because graceful degradation beats a total crash.

08 Quick Revision Sheet

Everything important on one page

Topic	Remember
Key metrics	Throughput, latency, response time, load/concurrency, utilization, business impact
Little's Law	$L = \lambda \times W$ (concurrency = throughput x response time)
Utilization rule	Aim for about 60-75% at peak; response time grows roughly as $1/(1 - \text{utilization})$
Lifecycle (slide 56)	Understand, Focus and design, Create, Implement and monitor, Evaluate and replan (a loop)
Six steps (slide 57)	Requirements, historical data, model the system, load test, provision with margin, estimate growth
Model the system	Workload modelling, resource consumption modelling, concurrency and scaling factors
Clustering gives	High availability, scalability, fault tolerance, resource management
Load balancing gives	Efficient utilization, better performance, redundancy, scalability
Clustering vs load balancing	Cluster = servers cooperating as one. Balancer = traffic distributor in front. Used together.
Algorithms	Round robin, weighted, least connections, least response time, hashing
Design rules	Stateless apps, health checks, N+1 capacity, redundant balancers, odd-sized quorum
Exam sentence	'Capacity estimation decides how much; clustering and load balancing deliver it reliably.'

Study tip

For interviews, practise the **worked example in Section 02** until you can do it from memory: daily requests, average RPS, peak RPS, usable capacity per server, round up, add N+1, then growth. Interviewers care more about the reasoning and stated assumptions than the final number.